

Optimizing a Specialized Transformer

Advisor: Hicham, mohamedhicham.leghettas@inf.ethz.ch

The geometric algebra transformer (GATr) [1] is a neural network architecture designed to process geometric data—such as 3D points, vectors, and planes. For this project, we can abstract away the underlying mathematics and view implementing GATr primarily as a structured computation problem. From an implementation standpoint, GATr functions like a standard transformer, but replaces the typical dot product with a geometric product, which is essentially a hardcoded “recipe” dictating how to multiply high dimensional arrays.

The Optimization Opportunity

In the existing PyTorch code¹, the geometric product is implemented using standard, dense operations (see for example `src/ezgatr/nn/functional/linear.py`, line 177). This approach is inefficient for two reasons:

- **Sparsity:** Many of the operations involve multiplying by zero.
- **Simplicity:** Most non-zero operations are basic sign-flips rather than floating-point operations.

Project Objective

The goal is to develop a high-performance C or C++ kernel for the components of GATr targeting x86_64 or ARM CPUs. By bypassing the overhead of general-purpose linear algebra libraries, you will create a solution optimized specifically for the unique structure of geometric algebra. Focusing strictly on inference, you can skip backward-pass logic and implement only the forward pass.

Implementation Details

The evaluation focuses on timing the forward function of the baseline `MVOnlyGATrModel` (see code snippet below) against the optimized `ASLMVOnlyGATrModel`. This requires swapping the following standard `ezgatr` components with their optimized counterparts:

- **Modules** (`ezgatr.nn`): `EquiLinear` and `EquiRMSNorm`.
- **Functions** (`ezgatr.nn.functional`): `equi_geometric_attention`, `equi_join`, `geometric_product`, and `scaler_gated_gelu`.

¹ <https://github.com/Guest400123064/ezgatr.git>

```
1 import torch
2 from ezgatr.nets.mv_only_gatr import MVOnlyGATrConfig, MVOnlyGATrModel, ASLMVOnlyGATrModel
3
4 device = "cpu" # only compare against the cpu versions, no cuda
5 size_channels_in = 2
6 batch_size = 8
7 config = MVOnlyGATrConfig(size_channels_in=size_channels_in)
8 x = torch.randn(batch_size, 256, size_channels_in, 16).to(device)
9
10 net = MVOnlyGATrModel(config).to(device)
11 output = net(x) # the baseline forward pass to be compared against
12
13 asl_net = ASLMVOnlyGATrModel(config).to(device)
14 output = asl_net(x) # the optimized version of the model
```

References

[1] Brehmer, J., De Haan, P., Behrends, S. and Cohen, T.S., 2023. Geometric algebra transformer. *Advances in Neural Information Processing Systems*, 36, pp.35472-35496